

Roteiro 2: Classificação supervisionada de intrusão

Relatório técnico

Antonio Almeida e Luka Figueiredo, Cibersegurança Aplicada com IA, Insper 2026-2

08/09/2026

1. Tabela-síntese do aquecimento

Dataset aquecimento_autenticacao.csv (3.160 IPs, 17,72% de ataque). Split 70/30 estratificado, random_state=3527138017.
Regra: falhas >= 10 AND usuarios_inexistentes >= 1.

Modelo	Acurácia	FN	FP	Recall ataque
Regra manual (teste)	0,9684	28	2	0,8333
Árvore de decisão (max_depth=3)	0,9979	2	0	0,9881
Regressão logística (com StandardScaler)	0,9905	8	1	0,9524
Regressão logística (sem scaler)	0,9989	1	0	0,9940

Features do 1º e 2º nível da árvore: usuarios_distintos (limiar 3,5) e usuarios_inexistentes (limiar 1,5). Não é a mesma regra do item b) (que usava falhas): usuarios_distintos responde por 95,5% do ganho de informação da árvore, contra 0,4% de falhas.

Sem StandardScaler, o número de iterações do solver saltou de **20 para 635** (~32x). A acurácia subiu (contraintuitivo); um teste adicional isolando escala e regularização (C=1 vs. C=1e6, com e sem scaler) mostrou que o ganho de acurácia vem da penalidade L2 atuando de forma desigual entre features de escalas diferentes, não da falta de padronização em si; o custo em iterações, esse sim, é sempre da escala.

2. Armadilhas encontradas no CICIDS2017 (dupla d23bda)

Dataset bruto: 111.679 fluxos x 83 colunas. Forma final após limpeza: **105.366 x 65**.

Passo	Linhas removidas	Linhas restantes
Bruto	-	111.679
Armadilha 2: rótulos impossíveis/contraditórios	223	111.456
Armadilha 3: duplicatas mascaradas	6.027	105.429
±Infinity e NaN	63	105.366

Colunas removidas: 18 (1 vazamento alertas_ids + 8 constantes + 9 redundantes).

Armadilha	Deteção	Impacto	Por que é problema	Tratamento
1. alertas_ids	Coluna fora do dicionário do Anexo A	1 coluna, 111.679 linhas	Corte único atinge 0,8648 de acurácia binária (baseline 0,5753); é a feature nº1 de 62 em importância num Random Forest completo	Removida antes de qualquer treino
2. Rótulos impossíveis/contraditórios	Confronto (dia, Label) com o calendário oficial do CICIDS2017 (Sharafaldin et al. 2018) + hash de vetores de features idênticos com rótulos distintos	184 linhas fora do calendário; 332 linhas em 146 grupos com vetor idêntico e rótulo diferente	Fluxo benigno duplicado e remarcado como ataque; contamina as classes DDoS/PortScan e quebraria o item I)	184 linhas fora do calendário removidas; nos grupos contraditórios, mantido o BENIGN original e removida a cópia rotulada como ataque (223 linhas no total)
3. Duplicatas mascaradas por coluna de contexto	duplicate() na linha inteira retorna 0 (impossível no CICIDS2017); reaparecem ao ignorar ordem_captura/alertas_ids	6.027 linhas excedentes em 1.440 grupos (assinatura de 54 colunas inteiras + rótulo)	Duplicatas atravessam o train_test_split: 5,9% do teste tinha gêmeo exato no treino antes da limpeza, com acurácia 1,0000 nesses casos contra 0,9896 no resto: memorização, não generalização	Deduplicado por features+Label, ordenado por (dia, ordem_captura), mantendo a ocorrência mais antiga

Armadilha mais perigosa (Q4): a 3 (duplicatas), não a 1. Removendo só o alertas_ids do modelo completo a acurácia cai apenas 0,08 pp (0,9895 para 0,9887): as outras 61 features cobrem a perda. Já manter as duplicatas eleva a acurácia de 0,9887 para 0,9902 (+0,15 pp), e a razão é direta: 1.976 fluxos de teste (5,9%) tinham cópia exata no treino, sempre acertados. O alertas_ids seria mentira por outro motivo: a coluna não existe no momento da decisão em produção, e qualquer seleção automática de features a escolheria de novo.

3. Tabela de separabilidade (item f)

Classe	Feature	Recall	Precisão	Separável?
FTP-Patator	Destination Port	1,0000	0,9343	Sim
Heartbleed	Bwd Packet Length Max	1,0000	1,0000	Sim
PortScan	Total Length of Fwd Packets	0,9953	0,5269	Sim
SSH-Patator	Destination Port	1,0000	0,8078	Sim
DoS Hulk	Total Length of Bwd Packets	0,8623	0,7035	Nao
DoS GoldenEye	Bwd Packet Length Std	0,6550	0,9867	Nao
DDoS	Total Length of Bwd Packets	0,6377	0,6922	Nao

Classe	Feature	Recall	Precisão	Separável?
DoS Slowhttptest (difícil)	Fwd IAT Mean	0,8047	0,5517	Nao
DoS slowloris (difícil)	Flow Packets/s	0,5786	0,5293	Nao
Infiltration	Total Length of Fwd Packets	0,2632	0,8333	Nao
Bot / Web Attack (x3)	Destination Port	0,0000	0,0000	Nao

Classes separáveis usam portas fixas ou artefatos de tamanho de pacote extremos. As não separáveis, incluindo a classe difícil, foram desenhadas para se misturar estatisticamente com tráfego HTTP/HTTPS legítimo (Q5).

5 features derivadas (item g), nenhuma separa a classe difícil isoladamente, mas 2 aparecem no top-15 de importância do Random Forest (item i): `razao_header_payload_fwd` (5a) e `bytes_por_pacote_fwd` (11a).

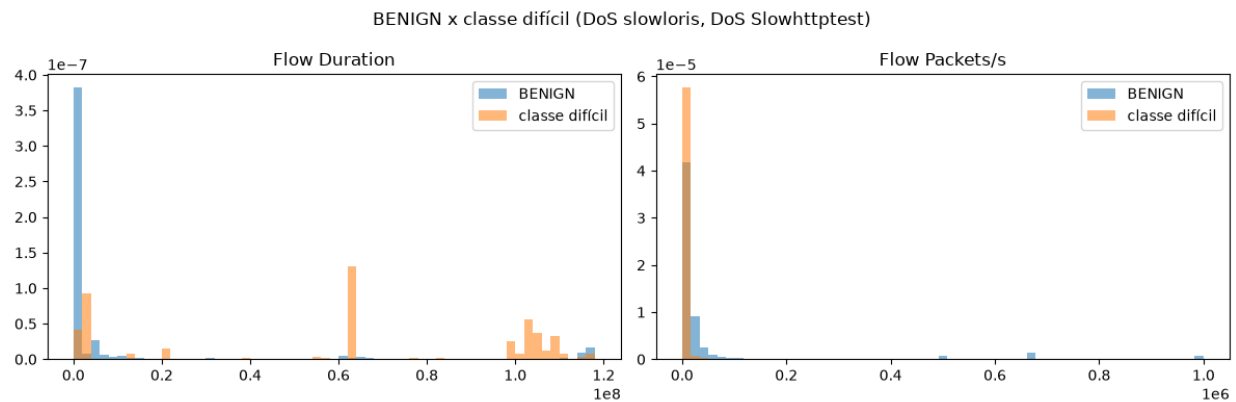


Figure 1: Histograma sobreposto BENIGN x classe difícil: sem corte unico que separe as duas distribuicoes

4. Recall aleatório x temporal (item h)

Random Forest (`n_estimators=200`), split aleatório 70/30 vs. split temporal por campanha (70% inicial / 30% final de cada par dia x rótulo, ordenado por `ordem_captura`).

Métrica	Split aleatório	Split temporal
Acurácia	0,9888	0,9407
Macro-F1	0,9053	0,8004

Classe	Recall aleatório	Recall temporal	Queda
Bot	0,9543	0,1486	0,8057
Heartbleed	1,0000	0,7500	0,2500
DoS slowloris (difícil)	0,9920	0,7904	0,2016
DoS Hulk	0,9955	0,8849	0,1106
DoS Slowhttptest (difícil)	0,9958	0,8879	0,1080
BENIGN	0,9979	0,9747	0,0232

Hipótese de rede para o Bot (maior queda, Q6): tráfego rápido no início da campanha (Flow Packets/s mediana ~1.046) vira quase ocioso no fim (~6 pacotes/s, duração de fluxo ~17x maior). O modelo aprendeu o padrão “rápido”, dominante no treino, e falha na fase tardia/silenciosa do mesmo bot.

Feature nova testada no item j ($\text{fracao_tempo_ocioso} = \text{Idle Mean} / \text{Flow Duration}$): **não ajudou de forma consistente** (Q8): recall de slowloris melhorou pouco (0,7904 para 0,7994), Slowhttpptest ficou igual (0,8879), macro-F1 quase não mudou (0,8004 → 0,8013), e a feature ficou na posição 55/67 de importância. Motivo: é uma recombinação linear de colunas que o modelo já tinha (Idle Mean, Flow Duration), redundância, não sinal novo.

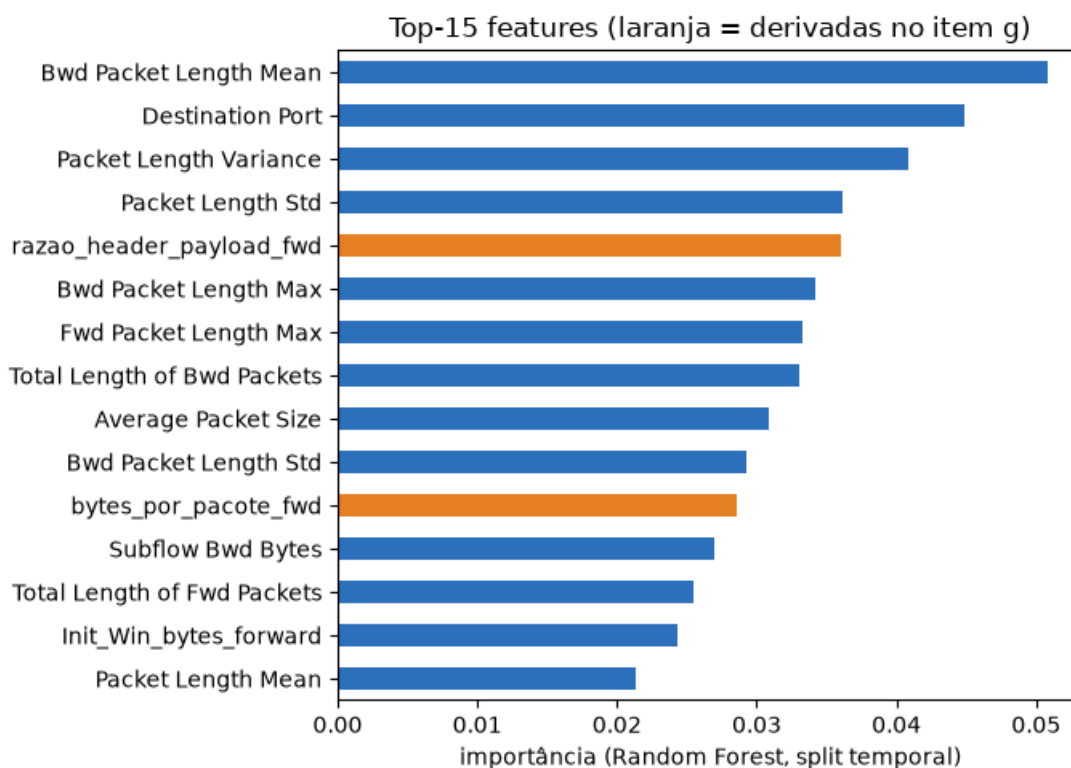


Figure 2: Top-15 features por importancia (Random Forest, split temporal): laranja = derivadas no item g

5. Parecer contra a produção

Cenário: um gestor viu a acurácia de 98,9% do split aleatório e quer colocar o modelo no IDS da empresa na segunda-feira. **Este parecer é contrário**, com cinco argumentos ancorados em números deste notebook:

1. **A acurácia de 98,9% mede memorização, não generalização.** O item e.10 mostrou diretamente que 1.976 fluxos de teste (5,9%) tinham gêmeo exato no treino antes da limpeza, com acurácia 1,0000 nesses casos contra 0,9896 nos demais.
2. **A avaliação honesta (split temporal) cai para 94,1% de acurácia e 0,80 de macro-F1**, contra 98,9% / 0,91 do split aleatório; a diferença só aparece quando o teste respeita a ordem do tempo, como o IDS vai operar de fato.
3. **O recall do Bot despenca de 95,4% para 14,9%** entre início e fim da mesma campanha de captura; o mesmo ataque, em fase mais silenciosa, escapa em mais de 85% dos casos.
4. **Contra ataques nunca vistos, o modelo não alarma.** No item l), 100% dos 1.971 fluxos de Bot na sexta-feira foram classificados como BENIGN, e 99,9% dos 8.161 fluxos de PortScan também.
5. **A coluna alertas_ids, mesmo custando só 0,08 pp quando removida, foi a feature nº1 de 62 em importância** quando deixada no modelo: qualquer seleção automática de features a escolheria de novo, reproduzindo o vazamento sem ninguém perceber.

Recomendação: avaliar sempre com split temporal e recall por classe (não só acurácia agregada), rodar em modo *shadow* antes de qualquer bloqueio automático, e reavaliar periodicamente. O item l) mostra que o modelo não tem nenhuma

defesa contra famílias de ataque inteiramente novas.

6. Ataques nunca vistos (item I)

Treino: segunda a quinta (66.751 fluxos). Teste: sexta-feira (38.615 fluxos). Graças à limpeza do item e (que removeu os fluxos fora do calendário oficial), Bot, DDoS e PortScan agora existem só na sexta-feira, exatamente como o enunciado descreve; todas as três são classes nunca vistas no treino.

Classe (sexta)	n	Para onde vai a previsão
Bot	1.971	100% -> BENIGN
PortScan	8.161	99,9% -> BENIGN
DDoS	8.578	63,4% -> DoS Hulk, 36,6% -> BENIGN, 0% -> DDoS
BENIGN	19.905	99,9% -> BENIGN (correto)

Acurácia geral na sexta: **51,5%**. Um IDS baseado em ML só enxerga o que existiu no treino: contra ameaça genuinamente nova ele fica em silêncio (Bot, PortScan -> BENIGN) ou dispara o alarme errado (DDoS -> DoS Hulk), nunca “detecta o ataque real”. Esse é exatamente o cenário mais importante para um IDS em produção: ataques novos são a razão de existir da detecção, não a exceção dela.

7. Respostas Q1-Q8

Q1. A árvore usou `usuarios_distintos` (1º nível, limiar 3,5) e `usuarios_inexistentes` (2º nível, limiar 1,5); não reinventou a regra manual, que usava falhas (que na árvore responde por só 0,4% do ganho). Colocaria a árvore em produção: acurácia de teste 0,9979 vs. 0,9684 da regra, com recall de ataque de 0,9881 contra 0,8333, e sem limiares escolhidos a dedo.

Q2. Sem `StandardScaler`, `n_iter_` saltou de 20 para 635 (~32x). O experimento isolando escala e regularização ($C=1$ vs. $C=1e6$) mostrou que o custo em iterações é sempre da amplitude de ~16.000x entre features (efeito de condicionamento), enquanto o ganho de acurácia observado vinha da penalidade L2 atuando desigualmente entre features de escalas diferentes quando não padronizadas: dois efeitos distintos, não um só.

Q3. `ip_origem` é identificador, não comportamento: há um IP por linha, então é uma chave única que um modelo pode memorizar (IP -> rótulo) sem aprender nenhum padrão de ataque. Em produção, todo IP observado é um valor nunca visto no treino, e o desempenho medido em treino viraria ficção.

Q4. As três armadilhas: `alertas_ids` (vazamento, feature nº1 de 62 em importância), 223 linhas com rótulos impossíveis/contraditórios (calendário + vetor idêntico com rótulo diferente), 6.027 quase-duplicatas mascaradas por `ordem_captura`. A mais perigosa é a 3: custa +0,15 pp de acurácia inflada (contra +0,08 pp do `alertas_ids`), e a evidência direta é 5,9% do teste com gêmeo exato no treino, sempre acertado.

Q5. Separáveis: FTP-Patator, SSH-Patator (porta fixa), Heartbleed (tamanho de pacote extremo, consequência do heap overflow), PortScan (volume de bytes por fluxo perto de zero). Não separáveis: Bot, DDoS, DoS GoldenEye/Hulk, Infiltration, Web Attack (x3) e a classe difícil, desenhados para se misturar estatisticamente com tráfego legítimo.

Q6. Bot, de recall 0,9543 para 0,1486. Evidência: `Flow Packets/s` cai de ~1.046 (início) para ~6 (fim) pacotes/s, `Flow Duration` cresce ~17x; o bot muda de tráfego rápido (varredura/exfiltração ativa) para quase-ocioso (heartbeat de C2) ao longo da campanha.

Q7. É a única avaliação honesta: em produção o modelo sempre classifica tráfego posterior ao treino, nunca vê o futuro durante o treino. O split aleatório permite vazamento de quase-duplicatas e padrões tardios para o treino, superestimando a generalização real: a queda de 98,9% para 94,1% de acurácia (e do recall do Bot, de 95,4% para 14,9%) só aparece quando essa ordem é respeitada.

Q8. Não ajudou de forma consistente (recall misto, macro-F1 quase parado em +0,0009, importância baixa, posição 55/67). Revela que uma razão calculada a partir de colunas que o modelo já usava (`Idle Mean`, `Flow Duration`) não injeta sinal novo. Diante de drift real, feature engineering sobre as mesmas medidas brutas tem limite; seria preciso sinal genuinamente novo ou retreinamento contínuo.

Anexo B · Ficha de resultados

Dupla: Antonio Almeida e Luka Figueiredo **Data:** 08/09/2026

Aquecimento

Modelo	Acurácia	FN	FP	Recall ataque
Regra manual (item b)	0,9684	28	2	0,833
Árvore d=3	0,9979	2	0	0,988
Reg. logística (com scaler)	0,9905	8	1	0,952
Reg. logística (sem scaler)	0,9989	1	0	0,994

Features do 1º e 2º nível da árvore: **usuarios_distintos, usuarios_inexistentes**

CICIDS2017

Código da dupla: **d23bda** random_state: **3527138017** classe difícil: **DoS lento (slowloris + Slowhttptest)**

Armadilha / passo de limpeza	Linhas (ou colunas) afetadas
Inf / NaN	63 linhas
Duplicatas exatas (linha inteira)	0 linhas
Armadilha 1: coluna alertas_ids fora do dicionário (vazamento; nº1/62 em importância)	1 coluna
Armadilha 2: rótulos impossíveis/contraditórios (calendário + vetor idêntico)	223 linhas
Armadilha 3: quase-duplicatas invisíveis a drop_duplicates()	6.027 linhas
Total final (linhas x colunas)	105.366 x 65

	Acurácia	Macro-F1	Recall classe difícil	Recall BENIGN
Split aleatório	0,9888	0,9053	0,992 / 0,996	0,998
Split temporal	0,9407	0,8004	0,790 / 0,888	0,975

Classes separáveis por uma feature: **FTP-Patator, SSH-Patator, Heartbleed, PortScan**

Classe que mais caiu no temporal: **Bot** de **0,954** para **0,149**

Feature do item j: **fracao_tempo_ocioso** efeito no recall temporal: **0,790 para 0,799 (slowloris) / 0,888 sem mudança (Slowhttptest);** nao ajudou de forma consistente